Task 2: Exploratory Data Analysis

Online Retail Dataset

This project performs exploratory data analysis on the cleaned Online Retail dataset to identify trends, patterns, relationships, and useful business insights.

Tools & Technologies

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Jupyter Notebook

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
```

In [2]:
```
df = pd.read_csv("cleaned_online_retail.csv")
```

In [3]:
```
df.head()
```

Out[3]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Cou
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	Un Kingc
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	Un Kingc
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	Un Kingc
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	Un Kingc
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	Un Kingc

In [4]:
```
df.shape
```

Out[4]: (541909, 8)

In [5]:
```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

```
Rows: 541909
Columns: 8
```

In [6]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype  
---  ------       --------------   -----  
 0   InvoiceNo    541909 non-null  object 
 1   StockCode    541909 non-null  object 
 2   Description  540455 non-null  object 
 3   Quantity     541909 non-null  int64  
 4   InvoiceDate  541909 non-null  object 
 5   UnitPrice    541909 non-null  float64
 6   CustomerID   406829 non-null  float64
 7   Country      541909 non-null  object 
dtypes: float64(2), int64(1), object(5)
memory usage: 33.1+ MB
```

In [7]:
```
df.columns
```

Out[7]:
```
Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity', 'InvoiceDate',
       'UnitPrice', 'CustomerID', 'Country'],
      dtype='object')
```

In [8]:
```
df.shape
```

Out[8]:
```
(541909, 8)
```

In [9]:
```
df.isnull().sum()
```

Out[9]:
```
InvoiceNo          0
StockCode          0
Description      1454
Quantity           0
InvoiceDate        0
UnitPrice          0
CustomerID     135080
Country            0
dtype: int64
```

In [10]:
```
df.duplicated().sum()
```

Out[10]:
```
np.int64(5268)
```

In [11]:
```
df.nunique()
```

Out[11]:
```
InvoiceNo      25900
StockCode       4070
Description     4223
Quantity         722
InvoiceDate    23260
UnitPrice       1630
CustomerID      4372
Country           38
dtype: int64
```

In [12]:
```
df.head()
```

Out[12]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Cou
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	Un Kingc
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	Un Kingc
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	Un Kingc
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	Un Kingc
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	Un Kingc

In [13]:
```
df.tail()
```

Out[13]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID
541904	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	2011-12-09 12:50:00	0.85	12680.0
541905	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	2011-12-09 12:50:00	2.10	12680.0
541906	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4	2011-12-09 12:50:00	4.15	12680.0
541907	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4	2011-12-09 12:50:00	4.15	12680.0
541908	581587	22138	BAKING SET 9 PIECE RETROSPOT	3	2011-12-09 12:50:00	4.95	12680.0

In [14]:
```
df.describe()
```

Out[14]:

	Quantity	UnitPrice	CustomerID
count	541909.000000	541909.000000	406829.000000
mean	9.552250	4.611114	15287.690570
std	218.081158	96.759853	1713.600303
min	-80995.000000	-11062.060000	12346.000000
25%	1.000000	1.250000	13953.000000
50%	3.000000	2.080000	15152.000000
75%	10.000000	4.130000	16791.000000
max	80995.000000	38970.000000	18287.000000

In [15]:
```
df.describe().T
```

Out[15]:

	count	mean	std	min	25%	50%	75%
Quantity	541909.0	9.552250	218.081158	-80995.00	1.00	3.00	10.00
UnitPrice	541909.0	4.611114	96.759853	-11062.06	1.25	2.08	4.13
CustomerID	406829.0	15287.690570	1713.600303	12346.00	13953.00	15152.00	16791.00

In [16]:
```
df.describe(include='object').T
```

Out[16]:

	count	unique	top	freq
InvoiceNo	541909	25900	573585	1114
StockCode	541909	4070	85123A	2313
Description	540455	4223	WHITE HANGING HEART T-LIGHT HOLDER	2369
InvoiceDate	541909	23260	2011-10-31 14:41:00	1114
Country	541909	38	United Kingdom	495478

In [17]:
```
df[['Quantity', 'UnitPrice']].describe()
```

Out[17]:

	Quantity	UnitPrice
count	541909.000000	541909.000000
mean	9.552250	4.611114
std	218.081158	96.759853
min	-80995.000000	-11062.060000
25%	1.000000	1.250000
50%	3.000000	2.080000
75%	10.000000	4.130000
max	80995.000000	38970.000000

In [18]:
```
df[['Quantity', 'UnitPrice']].median()
```

Out[18]:
```
Quantity     3.00
UnitPrice    2.08
dtype: float64
```

In [19]:
```
df['Revenue'] = df['Quantity'] * df['UnitPrice']
df[['Quantity', 'UnitPrice', 'Revenue']].head()
```

Out[19]:

	Quantity	UnitPrice	Revenue
0	6	2.55	15.30
1	6	3.39	20.34
2	8	2.75	22.00
3	6	3.39	20.34
4	6	3.39	20.34

In [20]:
```
total_revenue = df['Revenue'].sum()

print("Total Revenue:", total_revenue)
```

```
Total Revenue: 9747747.934
```

In [21]:
```
average_revenue = df['Revenue'].mean()

print("Average Revenue per Transaction:", average_revenue)
```

```
Average Revenue per Transaction: 17.987794876999644
```

In [22]:
```
median_revenue = df['Revenue'].median()

print("Median Revenue:", median_revenue)
```

```
Median Revenue: 9.75
```

In [23]:
```
print("Minimum Revenue:", df['Revenue'].min())
print("Maximum Revenue:", df['Revenue'].max())
```

```
Minimum Revenue: -168469.6
Maximum Revenue: 168469.6
```

In [24]:
```
df['Revenue'].describe()
```

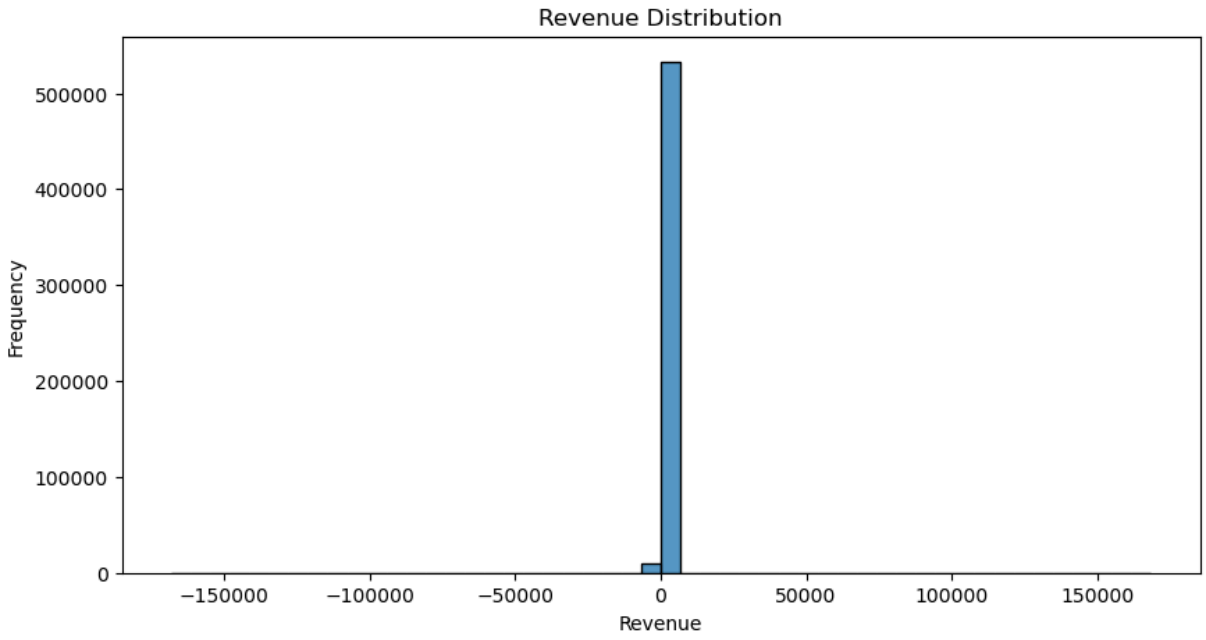
Out[24]:
```
count    541909.000000
mean         17.987795
std         378.810824
min     -168469.600000
25%           3.400000
50%           9.750000
75%          17.400000
max      168469.600000
Name: Revenue, dtype: float64
```

In [25]:
```
plt.figure(figsize=(10, 5))

sns.histplot(df['Revenue'], bins=50)

plt.title('Revenue Distribution')
plt.xlabel('Revenue')
plt.ylabel('Frequency')

plt.show()
```

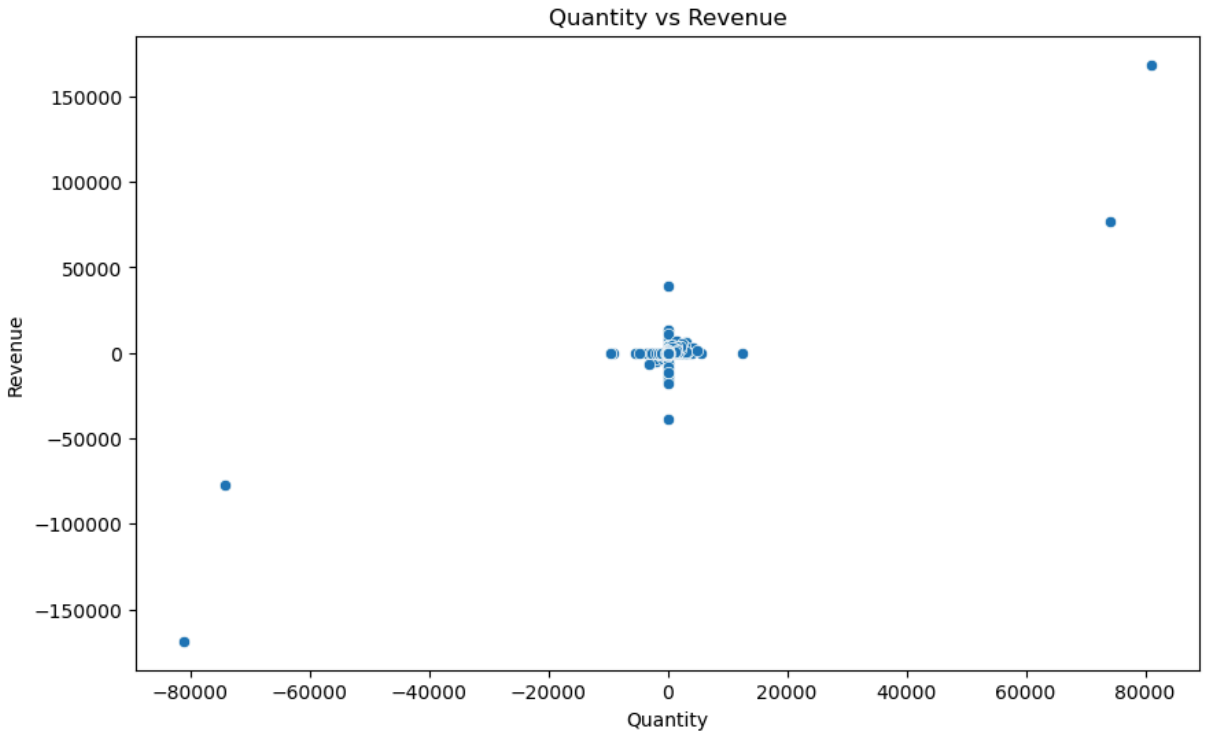


In [26]:
```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x='Quantity',
    y='Revenue'
)
```

```
plt.title('Quantity vs Revenue')
plt.xlabel('Quantity')
plt.ylabel('Revenue')

plt.show()
```



In [27]:
```
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])
```

In [28]:
```
df['InvoiceDate'].dtype
```

Out[28]:
```
dtype('<M8[ns]')
```

In [29]:
```
df['Year'] = df['InvoiceDate'].dt.year
df['Month'] = df['InvoiceDate'].dt.month
```

In [30]:
```
df['YearMonth'] = df['InvoiceDate'].dt.to_period('M')
```

In [31]:
```
df[['InvoiceDate', 'Year', 'Month', 'YearMonth']].head()
```

Out[31]:

	InvoiceDate	Year	Month	YearMonth
0	2010-12-01 08:26:00	2010	12	2010-12
1	2010-12-01 08:26:00	2010	12	2010-12
2	2010-12-01 08:26:00	2010	12	2010-12
3	2010-12-01 08:26:00	2010	12	2010-12
4	2010-12-01 08:26:00	2010	12	2010-12

In [32]:
```
monthly_sales = df.groupby('YearMonth')['Revenue'].sum().reset_index()

monthly_sales
```

Out[32]:

	YearMonth	Revenue
0	2010-12	748957.020
1	2011-01	560000.260
2	2011-02	498062.650
3	2011-03	683267.080
4	2011-04	493207.121
5	2011-05	723333.510
6	2011-06	691123.120
7	2011-07	681300.111
8	2011-08	682680.510
9	2011-09	1019687.622
10	2011-10	1070704.670
11	2011-11	1461756.250
12	2011-12	433668.010

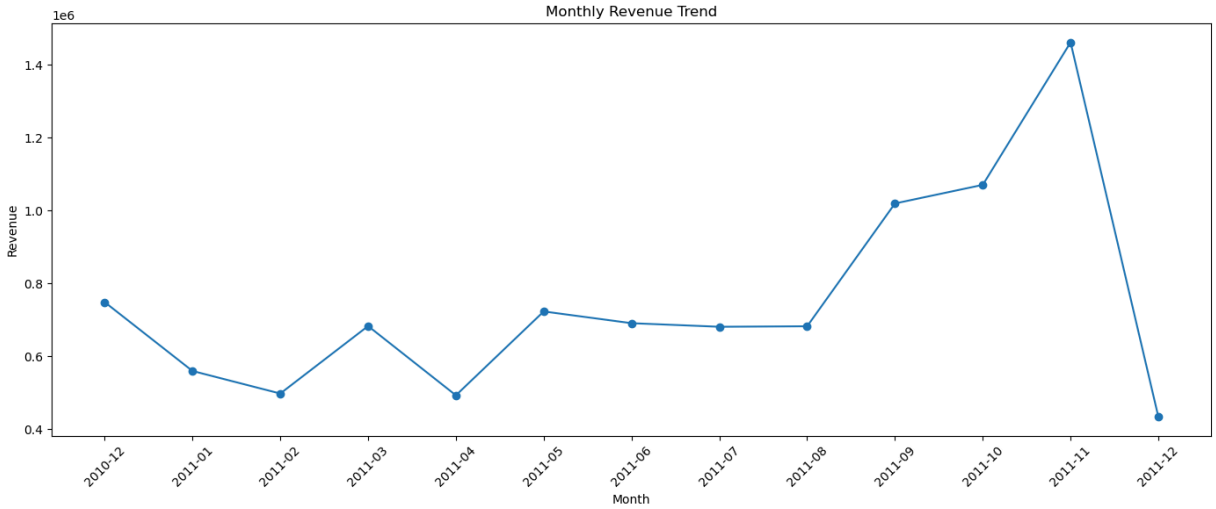
In [33]:
```
monthly_sales['YearMonth'] = monthly_sales['YearMonth'].astype(str)
```

In [34]:
```
plt.figure(figsize=(14, 6))

plt.plot(
    monthly_sales['YearMonth'],
    monthly_sales['Revenue'],
    marker='o'
)

plt.title('Monthly Revenue Trend')
plt.xlabel('Month')
plt.ylabel('Revenue')

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [35]:
```
highest_month = monthly_sales.loc[
    monthly_sales['Revenue'].idxmax()
]

highest_month
```

Out[35]:
```
YearMonth       2011-11
Revenue      1461756.25
Name: 11, dtype: object
```

In [36]:
```
lowest_month = monthly_sales.loc[
    monthly_sales['Revenue'].idxmin()
]

lowest_month
```

Out[36]:
```
YearMonth      2011-12
Revenue      433668.01
Name: 12, dtype: object
```

In [37]:
```
print("Highest Revenue Month:")
print(highest_month)

print("\nLowest Revenue Month:")
print(lowest_month)
```

```
Highest Revenue Month:
YearMonth       2011-11
Revenue      1461756.25
Name: 11, dtype: object

Lowest Revenue Month:
YearMonth      2011-12
Revenue      433668.01
Name: 12, dtype: object
```

In [38]:
```
top_products_revenue = (
    df.groupby('Description')['Revenue']
    .sum()
    .sort_values(ascending=False)
```

```
    .head(10)
)

top_products_revenue
```

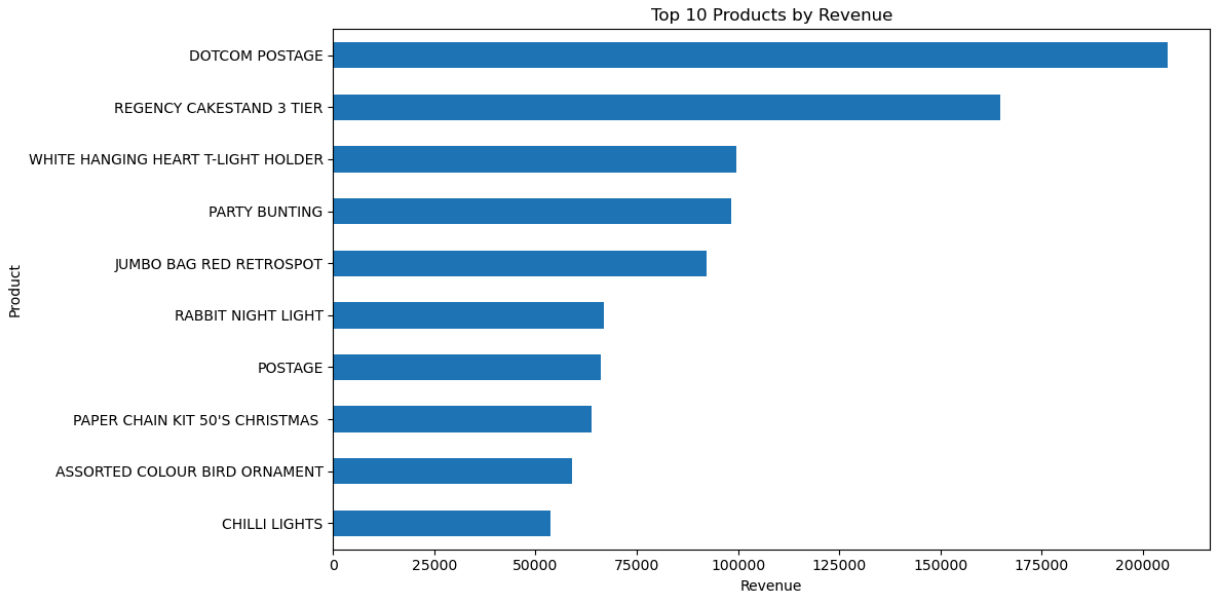
Out[38]:
```
Description
DOTCOM POSTAGE                        206245.48
REGENCY CAKESTAND 3 TIER              164762.19
WHITE HANGING HEART T-LIGHT HOLDER     99668.47
PARTY BUNTING                          98302.98
JUMBO BAG RED RETROSPOT                92356.03
RABBIT NIGHT LIGHT                     66756.59
POSTAGE                                66230.64
PAPER CHAIN KIT 50'S CHRISTMAS         63791.94
ASSORTED COLOUR BIRD ORNAMENT          58959.73
CHILLI LIGHTS                          53768.06
Name: Revenue, dtype: float64
```

In [39]:
```
plt.figure(figsize=(12, 6))

top_products_revenue.sort_values().plot(kind='barh')

plt.title('Top 10 Products by Revenue')
plt.xlabel('Revenue')
plt.ylabel('Product')

plt.tight_layout()
plt.show()
```



In [40]:
```
top_products_quantity = (
    df.groupby('Description')['Quantity']
    .sum()
    .sort_values(ascending=False)
    .head(10)
)

top_products_quantity
```

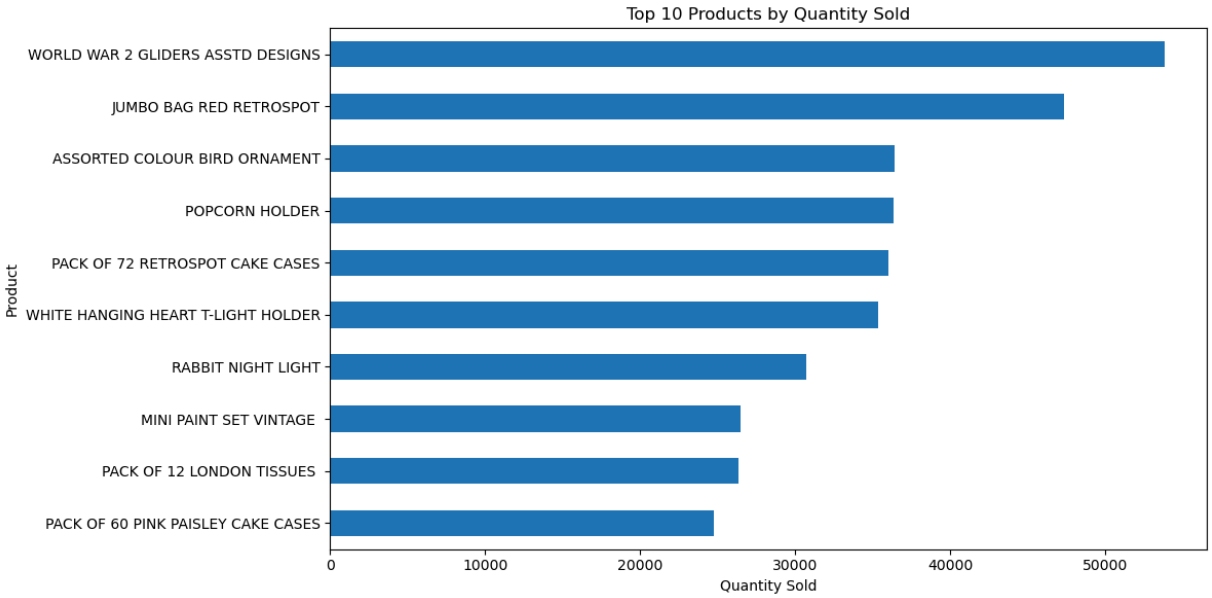
Out[40]:
```
Description
WORLD WAR 2 GLIDERS ASSTD DESIGNS     53847
JUMBO BAG RED RETROSPOT               47363
ASSORTED COLOUR BIRD ORNAMENT         36381
POPCORN HOLDER                        36334
PACK OF 72 RETROSPOT CAKE CASES       36039
WHITE HANGING HEART T-LIGHT HOLDER    35317
RABBIT NIGHT LIGHT                    30680
MINI PAINT SET VINTAGE                26437
PACK OF 12 LONDON TISSUES             26315
PACK OF 60 PINK PAISLEY CAKE CASES    24753
Name: Quantity, dtype: int64
```

In [41]:
```
plt.figure(figsize=(12, 6))

top_products_quantity.sort_values().plot(kind='barh')

plt.title('Top 10 Products by Quantity Sold')
plt.xlabel('Quantity Sold')
plt.ylabel('Product')

plt.tight_layout()
plt.show()
```



In [42]:
```
product_summary = (
    df.groupby('Description')
    .agg(
        Total_Quantity=('Quantity', 'sum'),
        Total_Revenue=('Revenue', 'sum')
    )
    .sort_values('Total_Revenue', ascending=False)
)

product_summary.head(10)
```

Out[42]:

Description	Total_Quantity	Total_Revenue
DOTCOM POSTAGE	707	206245.48
REGENCY CAKESTAND 3 TIER	13033	164762.19
WHITE HANGING HEART T-LIGHT HOLDER	35317	99668.47
PARTY BUNTING	18022	98302.98
JUMBO BAG RED RETROSPOT	47363	92356.03
RABBIT NIGHT LIGHT	30680	66756.59
POSTAGE	3003	66230.64
PAPER CHAIN KIT 50'S CHRISTMAS	18902	63791.94
ASSORTED COLOUR BIRD ORNAMENT	36381	58959.73
CHILLI LIGHTS	10229	53768.06

In [43]:
```
product_summary.head(10)
```

Out[43]:

Description	Total_Quantity	Total_Revenue
DOTCOM POSTAGE	707	206245.48
REGENCY CAKESTAND 3 TIER	13033	164762.19
WHITE HANGING HEART T-LIGHT HOLDER	35317	99668.47
PARTY BUNTING	18022	98302.98
JUMBO BAG RED RETROSPOT	47363	92356.03
RABBIT NIGHT LIGHT	30680	66756.59
POSTAGE	3003	66230.64
PAPER CHAIN KIT 50'S CHRISTMAS	18902	63791.94
ASSORTED COLOUR BIRD ORNAMENT	36381	58959.73
CHILLI LIGHTS	10229	53768.06

In [44]:
```
country_revenue = (
    df.groupby('Country')['Revenue']
    .sum()
    .sort_values(ascending=False)
)

country_revenue.head(10)
```

Out[44]:
```
Country
United Kingdom    8187806.364
Netherlands        284661.540
EIRE               263276.820
Germany            221698.210
France             197403.900
Australia          137077.270
Switzerland         56385.350
Spain               54774.580
Belgium             40910.960
Sweden              36595.910
Name: Revenue, dtype: float64
```

In [45]:
```
top_countries = country_revenue.head(10)

top_countries
```

Out[45]:
```
Country
United Kingdom    8187806.364
Netherlands        284661.540
EIRE               263276.820
Germany            221698.210
France             197403.900
Australia          137077.270
Switzerland         56385.350
Spain               54774.580
Belgium             40910.960
Sweden              36595.910
Name: Revenue, dtype: float64
```

In [46]:
```
top_countries = country_revenue.head(10)

top_countries
```

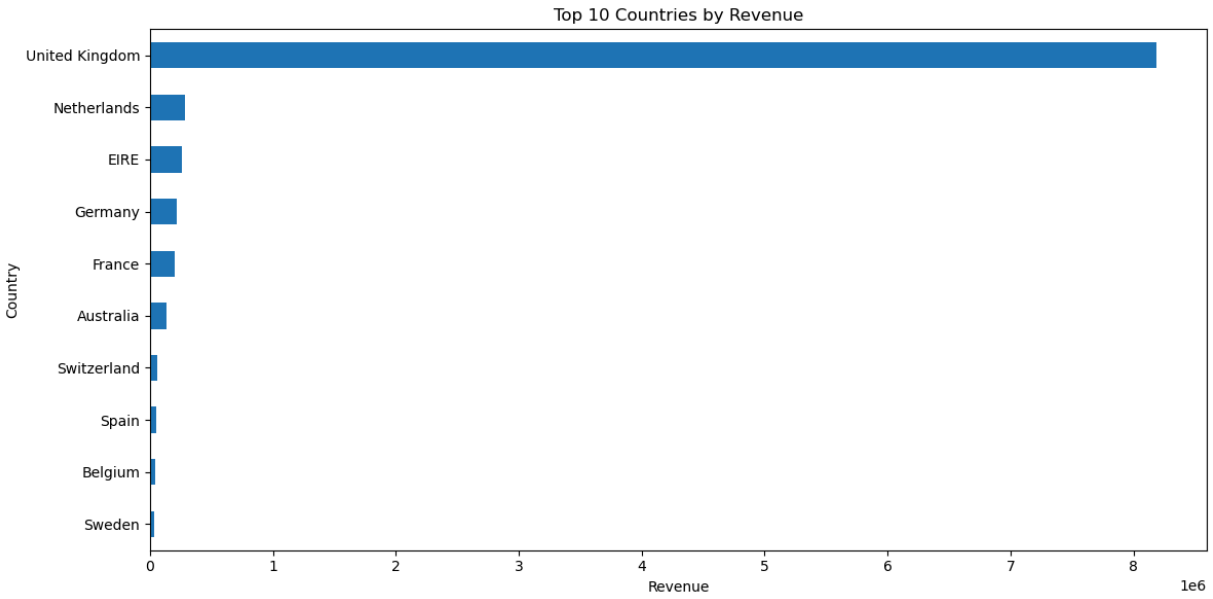
Out[46]:
```
Country
United Kingdom    8187806.364
Netherlands        284661.540
EIRE               263276.820
Germany            221698.210
France             197403.900
Australia          137077.270
Switzerland         56385.350
Spain               54774.580
Belgium             40910.960
Sweden              36595.910
Name: Revenue, dtype: float64
```

In [47]:
```
plt.figure(figsize=(12, 6))

top_countries.sort_values().plot(kind='barh')

plt.title('Top 10 Countries by Revenue')
plt.xlabel('Revenue')
plt.ylabel('Country')
```

```
plt.tight_layout()
plt.show()
```



In [48]:
```
country_quantity = (
    df.groupby('Country')['Quantity']
    .sum()
    .sort_values(ascending=False)
)

country_quantity.head(10)
```

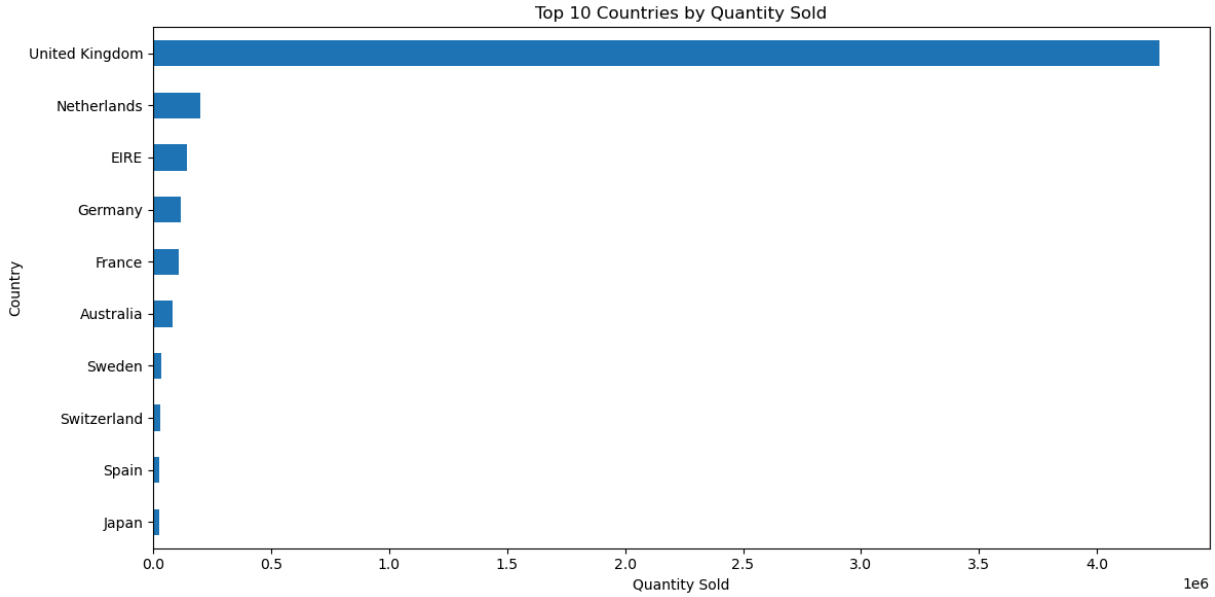
Out[48]:
```
Country
United Kingdom    4263829
Netherlands        200128
EIRE               142637
Germany            117448
France             110480
Australia           83653
Sweden              35637
Switzerland         30325
Spain               26824
Japan               25218
Name: Quantity, dtype: int64
```

In [49]:
```
plt.figure(figsize=(12, 6))

country_quantity.head(10).sort_values().plot(kind='barh')

plt.title('Top 10 Countries by Quantity Sold')
plt.xlabel('Quantity Sold')
plt.ylabel('Country')

plt.tight_layout()
plt.show()
```



In [50]:
```
country_transactions = (
    df.groupby('Country')['InvoiceNo']
    .nunique()
    .sort_values(ascending=False)
)

country_transactions.head(10)
```

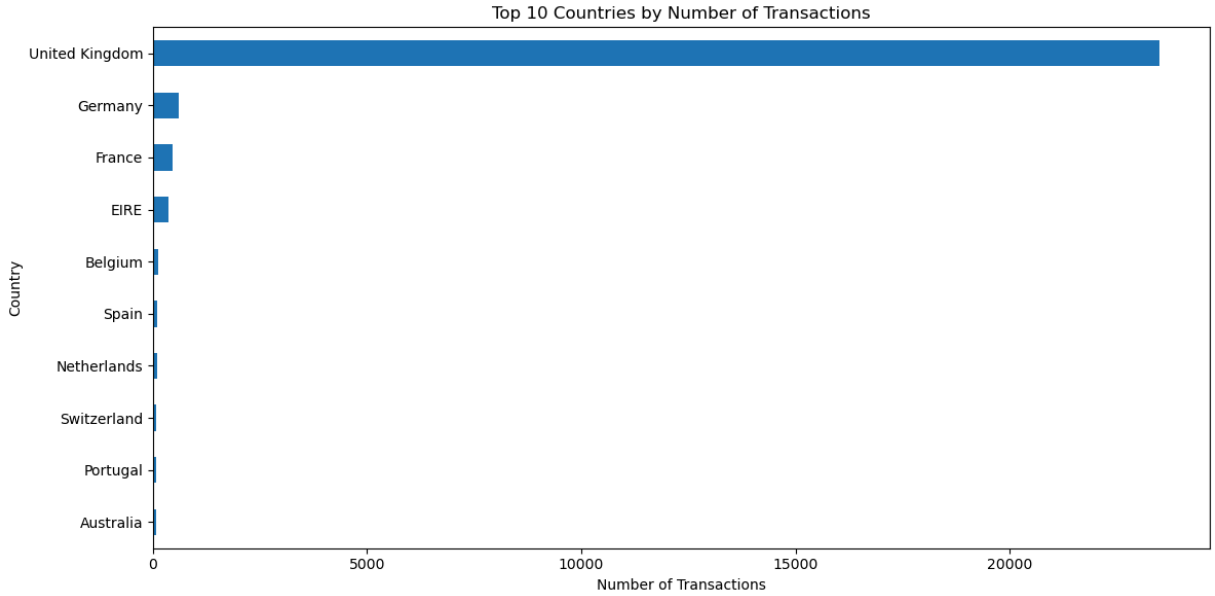
Out[50]:
```
Country
United Kingdom    23494
Germany             603
France              461
EIRE                360
Belgium             119
Spain               105
Netherlands         101
Switzerland          74
Portugal             71
Australia            69
Name: InvoiceNo, dtype: int64
```

In [51]:
```
plt.figure(figsize=(12, 6))

country_transactions.head(10).sort_values().plot(kind='barh')

plt.title('Top 10 Countries by Number of Transactions')
plt.xlabel('Number of Transactions')
plt.ylabel('Country')

plt.tight_layout()
plt.show()
```



In [52]:
```
country_summary = df.groupby('Country').agg(
    Total_Revenue=('Revenue', 'sum'),
    Total_Quantity=('Quantity', 'sum'),
    Total_Transactions=('InvoiceNo', 'nunique')
).sort_values('Total_Revenue', ascending=False)

country_summary.head(10)
```

Out[52]:

Country	Total_Revenue	Total_Quantity	Total_Transactions
United Kingdom	8187806.364	4263829	23494
Netherlands	284661.540	200128	101
EIRE	263276.820	142637	360
Germany	221698.210	117448	603
France	197403.900	110480	461
Australia	137077.270	83653	69
Switzerland	56385.350	30325	74
Spain	54774.580	26824	105
Belgium	40910.960	23152	119
Sweden	36595.910	35637	46

In [53]:
```
unique_customers = df['CustomerID'].nunique()

print("Total Unique Customers:", unique_customers)
```

```
Total Unique Customers: 4372
```

In [54]:
```
top_customers_revenue = (
    df.groupby('CustomerID')['Revenue']
    .sum()
    .sort_values(ascending=False)
    .head(10)
)

top_customers_revenue
```

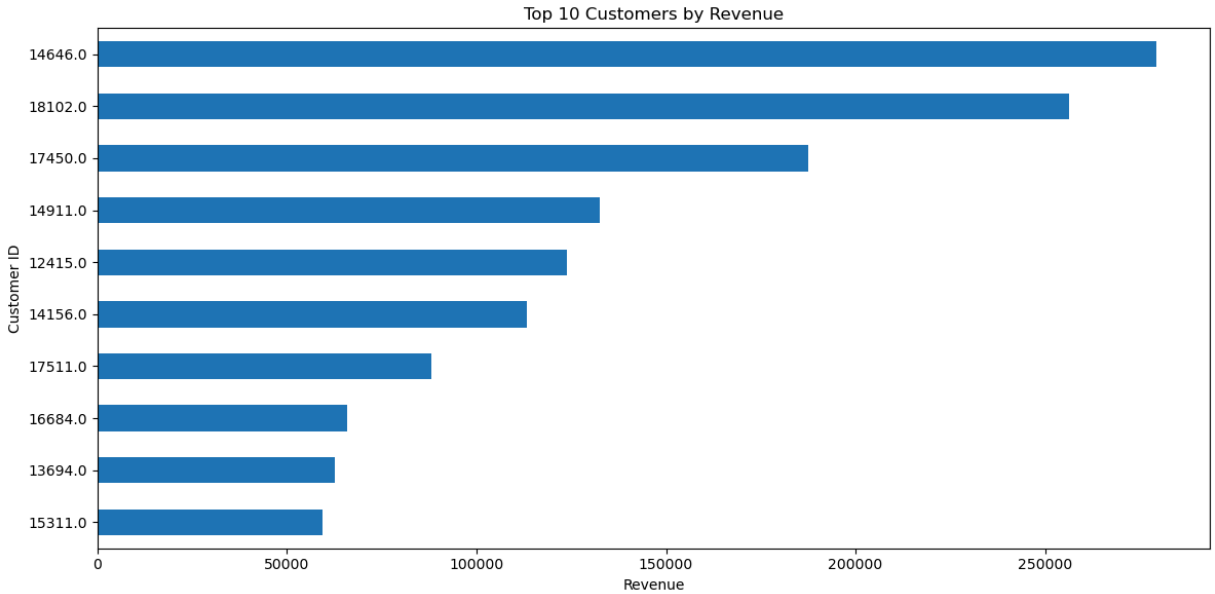
Out[54]:
```
CustomerID
14646.0    279489.02
18102.0    256438.49
17450.0    187482.17
14911.0    132572.62
12415.0    123725.45
14156.0    113384.14
17511.0     88125.38
16684.0     65892.08
13694.0     62653.10
15311.0     59419.34
Name: Revenue, dtype: float64
```

In [55]:
```
plt.figure(figsize=(12, 6))

top_customers_revenue.sort_values().plot(kind='barh')

plt.title('Top 10 Customers by Revenue')
plt.xlabel('Revenue')
plt.ylabel('Customer ID')

plt.tight_layout()
plt.show()
```



In [56]:
```
top_customers_orders = (
    df.groupby('CustomerID')['InvoiceNo']
    .nunique()
    .sort_values(ascending=False)
```

```
    .head(10)
)

top_customers_orders
```

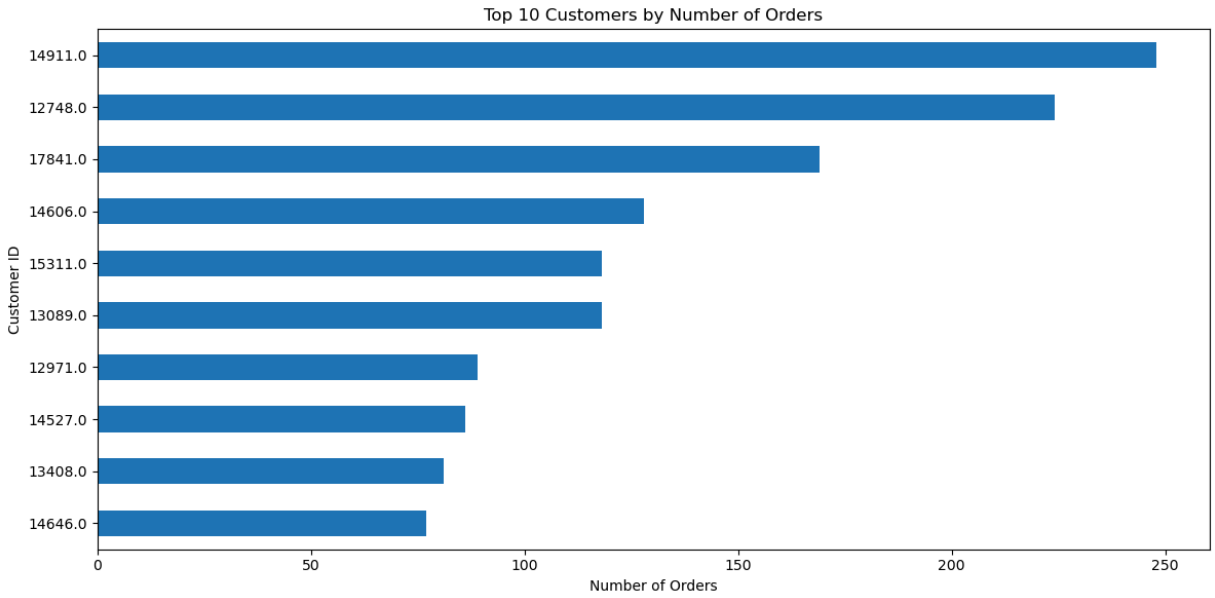
Out[56]:
```
CustomerID
14911.0    248
12748.0    224
17841.0    169
14606.0    128
13089.0    118
15311.0    118
12971.0     89
14527.0     86
13408.0     81
14646.0     77
Name: InvoiceNo, dtype: int64
```

In [57]:
```
plt.figure(figsize=(12, 6))

top_customers_orders.sort_values().plot(kind='barh')

plt.title('Top 10 Customers by Number of Orders')
plt.xlabel('Number of Orders')
plt.ylabel('Customer ID')

plt.tight_layout()
plt.show()
```



In [58]:
```
customer_summary = df.groupby('CustomerID').agg(
    Total_Revenue=('Revenue', 'sum'),
    Total_Orders=('InvoiceNo', 'nunique'),
    Total_Quantity=('Quantity', 'sum')
).sort_values('Total_Revenue', ascending=False)

customer_summary.head(10)
```

Out[58]:

CustomerID	Total_Revenue	Total_Orders	Total_Quantity
14646.0	279489.02	77	196719
18102.0	256438.49	62	64122
17450.0	187482.17	55	69029
14911.0	132572.62	248	77180
12415.0	123725.45	26	77242
14156.0	113384.14	66	57025
17511.0	88125.38	46	63012
16684.0	65892.08	31	49390
13694.0	62653.10	60	61803
15311.0	59419.34	118	37720

In [59]:
```
average_customer_revenue = customer_summary['Total_Revenue'].mean()

print("Average Revenue per Customer:", average_customer_revenue)
```

```
Average Revenue per Customer: 1898.4597012808783
```

In [60]:
```
median_customer_revenue = customer_summary['Total_Revenue'].median()

print("Median Revenue per Customer:", median_customer_revenue)
```

```
Median Revenue per Customer: 648.075
```

In [61]:
```
negative_quantity = df[df['Quantity'] < 0]

print("Negative Quantity Rows:", len(negative_quantity))
```

```
Negative Quantity Rows: 10624
```

In [62]:
```
print(
    "Percentage:",
    round(len(negative_quantity) / len(df) * 100, 2),
    "%"
)
```

```
Percentage: 1.96 %
```

In [63]:
```
negative_revenue = df[df['Revenue'] < 0]

print("Negative Revenue Rows:", len(negative_revenue))
```

```
Negative Revenue Rows: 9290
```

In [64]:
```
print(
    "Percentage:",
    round(len(negative_revenue) / len(df) * 100, 2),
```

```
    "%"
)
```

```
Percentage: 1.71 %
```

In [65]:
```
invalid_price = df[df['UnitPrice'] <= 0]

print("Zero/Negative UnitPrice Rows:", len(invalid_price))
```

```
Zero/Negative UnitPrice Rows: 2517
```

In [66]:
```
missing_customer = df['CustomerID'].isnull().sum()

print("Missing CustomerID:", missing_customer)
```

```
Missing CustomerID: 135080
```

In [67]:
```
print(
    "Percentage:",
    round(missing_customer / len(df) * 100, 2),
    "%"
)
```

```
Percentage: 24.93 %
```

In [68]:
```
cancelled = df[
    df['InvoiceNo'].astype(str).str.startswith('C')
]

print("Cancelled Transactions:", len(cancelled))
```

```
Cancelled Transactions: 9288
```

In [69]:
```
print(
    "Cancellation Percentage:",
    round(len(cancelled) / len(df) * 100, 2),
    "%"
)
```

```
Cancellation Percentage: 1.71 %
```

In [70]:
```
df.nlargest(10, 'Quantity')[
    ['InvoiceNo', 'Description', 'Quantity', 'UnitPrice', 'Revenue']
]
```

Out[70]:

	InvoiceNo	Description	Quantity	UnitPrice	Revenue
540421	581483	PAPER CRAFT , LITTLE BIRDIE	80995	2.08	168469.60
61619	541431	MEDIUM CERAMIC TOP STORAGE JAR	74215	1.04	77183.60
502122	578841	ASSTD DESIGN 3D PAPER STICKERS	12540	0.00	0.00
74614	542504	NaN	5568	0.00	0.00
421632	573008	WORLD WAR 2 GLIDERS ASSTD DESIGNS	4800	0.21	1008.00
206121	554868	SMALL POPCORN HOLDER	4300	0.72	3096.00
220843	556231	?	4000	0.00	0.00
97432	544612	EMPIRE DESIGN ROSETTE	3906	0.82	3202.92
270885	560599	ESSENTIAL BALM 3.5g TIN IN ENVELOPE	3186	0.06	191.16
52711	540815	FAIRY CAKE FLANNEL ASSORTED COLOUR	3114	2.10	6539.40

In [71]:
```
df.nlargest(10, 'UnitPrice')[
    ['InvoiceNo', 'Description', 'Quantity', 'UnitPrice', 'Revenue']
]
```

Out[71]:

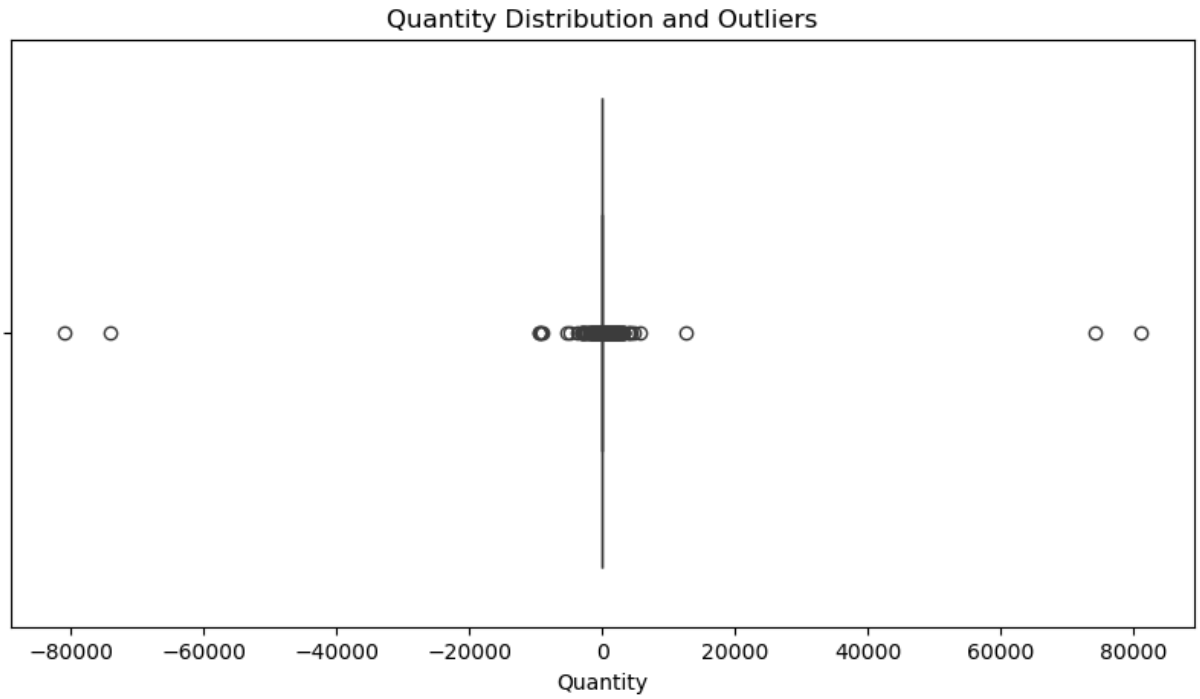
	InvoiceNo	Description	Quantity	UnitPrice	Revenue
222681	C556445	Manual	-1	38970.00	-38970.00
524602	C580605	AMAZON FEE	-1	17836.46	-17836.46
43702	C540117	AMAZON FEE	-1	16888.02	-16888.02
43703	C540118	AMAZON FEE	-1	16453.71	-16453.71
15016	C537630	AMAZON FEE	-1	13541.33	-13541.33
15017	537632	AMAZON FEE	1	13541.33	13541.33
16356	C537651	AMAZON FEE	-1	13541.33	-13541.33
16232	C537644	AMAZON FEE	-1	13474.79	-13474.79
524601	C580604	AMAZON FEE	-1	11586.50	-11586.50
299982	A563185	Adjust bad debt	1	11062.06	11062.06

In [72]:
```
plt.figure(figsize=(10, 5))

sns.boxplot(x=df['Quantity'])

plt.title('Quantity Distribution and Outliers')
plt.xlabel('Quantity')
```

```
plt.show()
```

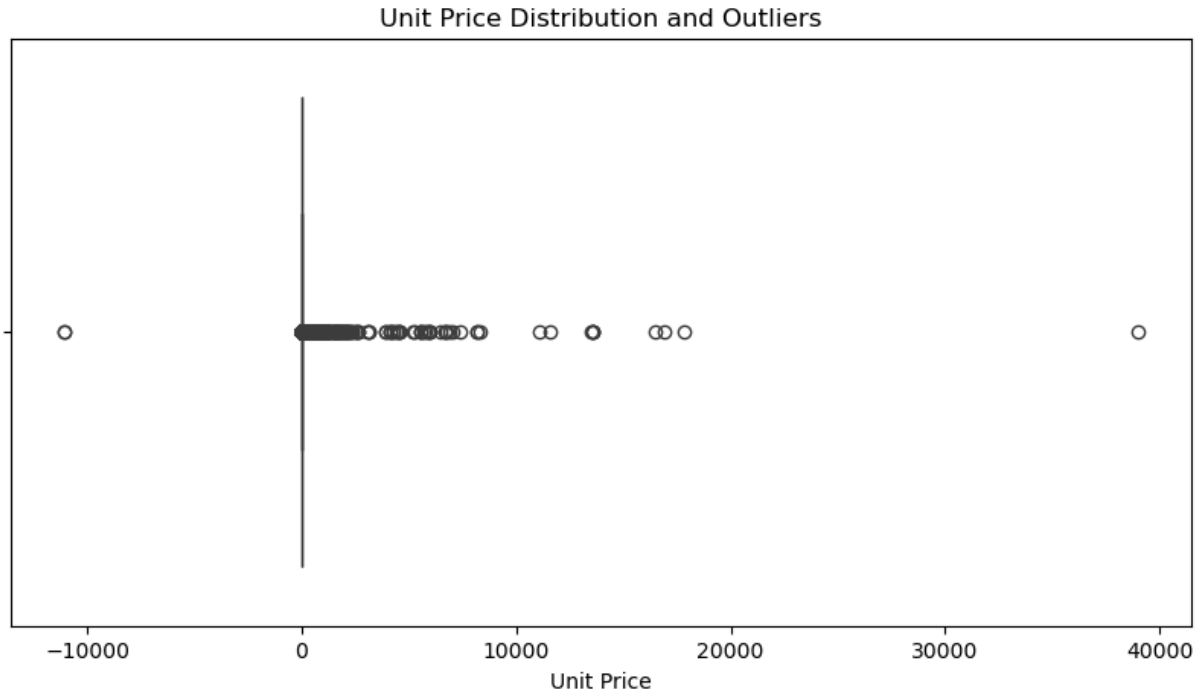


In [73]:
```
plt.figure(figsize=(10, 5))

sns.boxplot(x=df['UnitPrice'])

plt.title('Unit Price Distribution and Outliers')
plt.xlabel('Unit Price')

plt.show()
```



Key Business Insights

Based on the exploratory data analysis, the following key insights were identified:

In [74]:
```
print("Total Revenue:", round(df['Revenue'].sum(), 2))
print("Average Revenue per Transaction:", round(df['Revenue'].mean(), 2))
print("Median Revenue per Transaction:", round(df['Revenue'].median(), 2))
print("Unique Customers:", df['CustomerID'].nunique())
print("Unique Countries:", df['Country'].nunique())
print("Total Transactions:", df['InvoiceNo'].nunique())
```

```
Total Revenue: 9747747.93
Average Revenue per Transaction: 17.99
Median Revenue per Transaction: 9.75
Unique Customers: 4372
Unique Countries: 38
Total Transactions: 25900
```

In [75]:
```
print("Top Revenue Country:")
print(country_summary.head(1))
```

```
Top Revenue Country:
                Total_Revenue  Total_Quantity  Total_Transactions
Country
United Kingdom    8187806.364         4263829               23494
```

In [76]:
```
print("Top Revenue Product:")
print(product_summary.head(1))
```

```
Top Revenue Product:
                Total_Quantity  Total_Revenue
Description
DOTCOM POSTAGE             707      206245.48
```

In [77]:
```
print("Top Customer by Revenue:")
print(customer_summary.head(1))
```

```
Top Customer by Revenue:
            Total_Revenue  Total_Orders  Total_Quantity
CustomerID
14646.0         279489.02            77          196719
```

In [78]:
```
print("Highest Revenue Month:")
print(highest_month)
```

```
Highest Revenue Month:
YearMonth       2011-11
Revenue      1461756.25
Name: 11, dtype: object
```

In [79]:
```
print("========== FINAL EDA SUMMARY ==========")

print("\nTotal Rows:", len(df))
print("Total Columns:", len(df.columns))
print("Total Revenue:", round(df['Revenue'].sum(), 2))
```

```
print("Unique Customers:", df['CustomerID'].nunique())
print("Unique Countries:", df['Country'].nunique())
print("Unique Invoices:", df['InvoiceNo'].nunique())

print("\nTop Revenue Country:")
print(country_summary.index[0])

print("\nTop Revenue Product:")
print(product_summary.index[0])

print("\nTop Revenue Customer:")
print(customer_summary.index[0])

print("\nHighest Revenue Month:")
print(highest_month['YearMonth'])

print("\nLowest Revenue Month:")
print(lowest_month['YearMonth'])
```

```
========== FINAL EDA SUMMARY ==========

Total Rows: 541909
Total Columns: 12
Total Revenue: 9747747.93
Unique Customers: 4372
Unique Countries: 38
Unique Invoices: 25900

Top Revenue Country:
United Kingdom

Top Revenue Product:
DOTCOM POSTAGE

Top Revenue Customer:
14646.0

Highest Revenue Month:
2011-11

Lowest Revenue Month:
2011-12
```

Key Business Insights

1. Sales Performance

The analysis shows that the business generated significant revenue across the analyzed period. The monthly revenue trend indicates variations in sales performance across different months.

2. Product Performance

A small group of products contributes a significant portion of the overall revenue, indicating that certain products are particularly important to business performance.

3. Geographic Performance

The country-level analysis shows that some markets contribute substantially more revenue than others, highlighting the importance of focusing on high-performing markets.

4. Customer Performance

The customer analysis shows that some customers generate considerably higher revenue than others, indicating the presence of high-value customers.

5. Data Quality and Anomalies

The dataset contains unusual observations such as negative quantities, cancelled transactions, missing customer IDs, and extreme values. These should be considered when performing further business analysis.

Conclusion

The exploratory data analysis provided an overall understanding of sales performance, product behavior, customer activity, geographic performance, and data quality.

The analysis identified important trends, high-performing products and markets, valuable customers, and potential anomalies in the dataset. These findings can be used as a foundation for further business analysis, visualization, and predictive modeling.

In [ ]: